

# RISK ANALYSIS

## BCanteen: Digital Foodcourt Ordering Platform

### 1. Introduction

Risk analysis is an essential component of the software development process. It involves identifying potential problems that could affect the project, assessing the likelihood and impact of those problems, and defining strategies to reduce or eliminate them. In the context of BCanteen, a web-based digital foodcourt ordering platform, risk analysis helps ensure that the system is reliable, secure, and maintainable from development through deployment. This document covers four main areas of risk: risk identification, security risks, technical risks, and mitigation strategies. It also describes how testing activities are integrated throughout the development process to detect and resolve issues at each phase.

### 2. Risk Identification

The following table identifies known risks in the BCanteen system, along with their assessed impact level, probability of occurrence, and proposed mitigation approach.

| Risk                                      | Impact | Probability | Mitigation Strategy                                                                                                                                           |
|-------------------------------------------|--------|-------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Server down / backend unavailable         | High   | Medium      | Implement global error handler middleware; display user-friendly error messages on the frontend; use Axios interceptors to catch network failures gracefully. |
| Invalid or malformed order data submitted | High   | Low         | Apply backend validation using express-validator; reject requests with missing or invalid fields before they reach the database.                              |
| Duplicate order code generation           | High   | Low         | Use database-level UNIQUE constraint on order_code column; wrap order creation in a DB transaction to prevent race conditions.                                |
| Database connection failure               | High   | Low         | Configure connection pooling with retry logic; return 503 status code when DB is unreachable; log errors for monitoring.                                      |
| Cart data lost on page refresh            | Medium | High        | Persist cart state to localStorage via Pinia plugin; restore cart on app initialization.                                                                      |

|                                        |        |        |                                                                                                                                       |
|----------------------------------------|--------|--------|---------------------------------------------------------------------------------------------------------------------------------------|
| Incorrect total price calculation      | High   | Low    | Calculate subtotal and total on the backend, not the frontend; validate that frontend-submitted totals match backend-computed values. |
| Menu item unavailable at checkout      | Medium | Medium | Check <code>is_available</code> flag on all menu items during order creation on the backend; return error if any item is unavailable. |
| CORS policy blocking API requests      | Medium | Medium | Configure cors middleware in Express to explicitly allow the frontend origin; restrict allowed methods and headers.                   |
| API endpoint exposed to abuse          | Medium | High   | Apply rate limiting middleware ( <code>express-rate-limit</code> ) on all API routes; validate request body size and content type.    |
| SQL injection via user input           | High   | Low    | Use parameterized queries (prepared statements) via <code>mysql2</code> ; never concatenate user input directly into SQL strings.     |
| Sensitive data exposed in API response | Medium | Low    | Audit all API responses; never return internal IDs, raw DB errors, or stack traces to the client in production.                       |
| Frontend-backend data type mismatch    | Medium | Medium | Define and document API contract; parse and validate response data types in Axios service layer before use in Vue components.         |

### 3. Security Risks

Although BCanteen is an MVP without user authentication, several security risks still apply at the API and database level. The following subsections describe each identified security risk and the countermeasure applied.

#### 3.1 SQL Injection

SQL injection is an attack in which malicious SQL code is inserted into an input field and executed against the database. In BCanteen, all database queries use parameterized queries through the `mysql2` library. This means user-supplied values are always passed as separate parameters and never concatenated directly into SQL strings, which completely prevents this class of attack.

Example of safe query implementation: `db.query("SELECT * FROM menus WHERE tenant_id = ?", [tenantId])`

### **3.2 Cross-Origin Resource Sharing (CORS) Issues**

CORS is a browser security mechanism that blocks HTTP requests made from a different origin than the server. Without proper CORS configuration, the Vue frontend (running on port 5173) would be blocked from calling the Express API (running on port 3000). BCanteen mitigates this by configuring the cors middleware in Express to explicitly whitelist the frontend origin. In production, only the deployed frontend URL is allowed.

- Development: `VITE_API_BASE_URL=http://localhost:3000` is allowed
- Production: Only the deployed frontend domain is whitelisted in CORS config
- Credentials and non-standard headers are explicitly restricted

### **3.3 API Abuse and Excessive Requests**

Without any form of rate limiting, the BCanteen API is vulnerable to abuse either through automated scripts flooding the order endpoint or denial-of-service attempts. The mitigation strategy involves applying the `express-rate-limit` middleware, which limits the number of requests a single IP address can make within a given time window.

- `POST /api/orders` is rate-limited to prevent order flooding
- All API routes apply a general rate limit of 100 requests per 15 minutes per IP
- Requests exceeding the limit receive a 429 Too Many Requests response

### **3.4 Exposed Internal Error Details**

Returning raw database errors or stack traces in API responses can expose internal system details to attackers. BCanteen uses a centralized error handler middleware that intercepts all unhandled errors, logs them server-side, and returns only a generic error message to the client. In production mode, no stack trace is included in the response.

## **4. Technical Risks**

Beyond security concerns, several technical risks were identified during the design and development of BCanteen. These relate to system stability, data integrity, and the reliability of communication between the frontend and backend.

### **4.1 API Failure**

The Vue frontend is fully dependent on the Express backend API. If any API endpoint fails due to a server error, unhandled exception, or database issue the user experience is directly affected. To mitigate this, every API route is wrapped in a try-catch block, and all errors are forwarded to the centralized error handler. On the frontend, Axios response interceptors catch

non-2xx responses and display appropriate error states to the user rather than silently failing or crashing the application.

## **4.2 Network Latency**

Network latency between the frontend and backend can cause the UI to appear unresponsive, especially during data fetching on pages like the foodcourt list or tenant menu. BCanteen addresses this by implementing loading spinner states on all data-fetching operations. API calls are initiated as soon as the component mounts (using Vue's `onMounted` lifecycle hook), and the UI renders a skeleton or loading indicator while data is being retrieved.

## **4.3 Frontend-Backend Data Mismatch**

A common integration risk in REST API applications is a mismatch between the data structure expected by the frontend and what the backend actually returns. For example, if the backend returns price as a string but the frontend treats it as a number, calculation errors will occur. BCanteen mitigates this through a clearly defined API contract documented before implementation begins, strict data type handling in the Axios service layer, and backend-computed totals that the frontend displays rather than recalculates independently.

## **4.4 Cart State Inconsistency**

Because cart state is managed in-memory via Pinia, a page refresh would normally clear all cart data. This is a significant usability risk. The mitigation is to persist the cart store to `localStorage` so that items survive page reloads. Additionally, the cart enforces a single-tenant rule: if a user attempts to add an item from a different tenant than the current cart contents, a confirmation dialog prompts them to clear the existing cart first, preventing cross-tenant data mixing.

## **4.5 Order Code Collision**

If two orders are created simultaneously, there is a theoretical risk that both could receive the same generated order code. BCanteen addresses this with two layers of protection: a `UNIQUE` constraint on the `order_code` column at the database level, and wrapping the code generation and order insertion within a single database transaction. If a collision occurs, the transaction will fail and be retried, ensuring uniqueness is always maintained.

# **5. Mitigation Strategies**

The following section summarizes the technical mitigation strategies applied in the BCanteen system, organized by layer.

## **5.1 Backend Validation (express-validator)**

All incoming API requests that accept user-supplied data are validated using the `express-validator` library before reaching the business logic layer. Validation rules include field

presence checks, data type enforcement, and value range constraints. If validation fails, a 400 Bad Request response is returned with a descriptive error message indicating which fields are invalid.

Validation is applied to the following endpoints:

- POST /api/orders: validates tenant\_id, items array, item quantities, and total\_price
- Query parameters on GET endpoints: validates that numeric IDs are positive integers
- Search query parameters: sanitized to prevent injection via query strings

## 5.2 Centralized Error Handler Middleware

A global error handler middleware is registered as the last middleware in the Express application. All errors thrown or passed via next(error) anywhere in the application are caught by this handler. It standardizes the error response format, logs the error details server-side, and ensures the client always receives a consistent JSON error object without internal implementation details.

Standard error response format returned to the client: `{ "success": false, "message": "error description", "code": 400 }`

## 5.3 Database Transactions

Order creation involves writing to two tables simultaneously: orders and order\_items. To ensure data integrity, this operation is wrapped in a database transaction. If any part of the operation fails for example, if inserting an order\_item fails after the order record has been created the entire transaction is rolled back, leaving the database in a consistent state. This prevents orphaned order records or partially created orders from appearing in the system.

## 5.4 Input Sanitization

In addition to validation, all string inputs are sanitized to remove potentially harmful characters. The express-validator library provides sanitizer functions that trim whitespace, escape HTML special characters, and normalize data before it is processed or stored. This adds a secondary layer of defense against injection attacks.

## 5.5 Rate Limiting

The express-rate-limit middleware is applied to protect the API from abusive request patterns. A general rate limit is applied to all routes, and a stricter limit is applied specifically to the POST /api/orders endpoint to prevent order flooding. When the limit is exceeded, the server returns a 429 status code with a message informing the client to retry after a defined time window.

# 6. Testing Activities

Testing was integrated throughout the BCanteen development process, not treated as a single end-of-project activity. Each phase of development included specific testing activities appropriate to the increment being delivered.

## 6.1 Unit Testing

Unit testing focused on individual functions and modules in isolation, particularly the order code generator utility and the price calculation logic. These components were verified to produce correct outputs for a range of valid and edge-case inputs before being integrated into the broader system.

- Order code generator tested for sequential output, zero-padding, and prefix format
- Price calculation tested with single items, multiple items, and zero-quantity edge cases
- Validation rules tested individually with valid, missing, and out-of-range inputs

## 6.2 Integration Testing

Integration testing verified that the individual modules of the backend worked correctly together. Specifically, this included testing that the controller correctly called the service layer, the service correctly queried the database, and the response matched the documented API contract. API endpoints were tested using manual HTTP requests during Phase 3 (Backend Core) before the frontend was connected.

- GET /api/foodcourts verified to return correct JSON structure with all required fields
- GET /api/tenants/:id/menus verified to filter correctly by category parameter
- POST /api/orders verified to create order record, order\_items records, and return order code
- Foreign key constraints verified by attempting to insert invalid tenant\_id values

## 6.3 System Testing

System testing was conducted after the frontend and backend were fully integrated in Phase 5. The complete customer journey was tested end-to-end: from opening the landing page, browsing foodcourts and tenants, adding items to the cart, proceeding through checkout, and receiving the generated order code on the success page.

| Test Scenario                              | Expected Result                              | Phase Tested | Result |
|--------------------------------------------|----------------------------------------------|--------------|--------|
| Load home page with featured foodcourts    | Foodcourt cards rendered with correct data   | Phase 4      | Pass   |
| Navigate to foodcourt and view tenant list | Tenants listed with open/closed status       | Phase 4      | Pass   |
| Browse tenant menu grouped by category     | Menu items displayed with price in Rp format | Phase 4      | Pass   |

|                                              |                                                  |         |      |
|----------------------------------------------|--------------------------------------------------|---------|------|
| Add item to cart from tenant menu            | Cart count and total updated in view cart bar    | Phase 5 | Pass |
| Attempt to add item from different tenant    | Conflict prompt appears; cart clears on confirm  | Phase 5 | Pass |
| Update item quantity in cart                 | Total price recalculates correctly               | Phase 5 | Pass |
| Proceed to checkout and review order summary | Correct subtotal, tax (10%), and total displayed | Phase 5 | Pass |
| Place order by clicking Place Order button   | Order code generated in BCA-XXXX format          | Phase 5 | Pass |
| View order success page                      | Order code, items, total, status Pending shown   | Phase 5 | Pass |
| Submit order with backend disconnected       | User-friendly error message displayed            | Phase 6 | Pass |

## 6.4 Acceptance Testing

Acceptance testing confirmed that the completed system satisfied the original project requirements as defined in the MVP specification. Each business rule and required feature was verified against the implemented system.

| Acceptance Criterion                                              | Verification Method      | Status   |
|-------------------------------------------------------------------|--------------------------|----------|
| Customer can browse foodcourts without registering or logging in  | Manual walkthrough       | Accepted |
| Customer can add items to cart and update quantities              | Manual walkthrough       | Accepted |
| Cart enforces single-tenant rule with conflict prompt             | Manual walkthrough       | Accepted |
| System generates unique order code in BCA-XXXX format             | Inspect DB + UI output   | Accepted |
| Order is stored in database with status Pending by default        | SQL query verification   | Accepted |
| Checkout calculates correct total including 10% tax               | Manual calculation check | Accepted |
| Order success page displays order code and full order summary     | Manual walkthrough       | Accepted |
| API returns correct HTTP status codes for success and error cases | HTTP response inspection | Accepted |

|                                                                    |                   |          |
|--------------------------------------------------------------------|-------------------|----------|
| Application displays appropriate error when backend is unavailable | Backend kill test | Accepted |
|--------------------------------------------------------------------|-------------------|----------|

## 7. Conclusion

Risk analysis for BCanteen identified twelve distinct risks spanning security, data integrity, system stability, and frontend-backend integration. For each risk, a concrete mitigation strategy was defined and implemented within the system, including parameterized queries against SQL injection, CORS configuration, rate limiting against API abuse, database transactions for order integrity, and centralized error handling for graceful failure recovery. Testing activities were integrated across all six development phases rather than deferred to the end of the project. Unit tests validated individual utilities, integration tests confirmed API correctness before frontend connection, system tests verified the complete end-to-end customer journey, and acceptance tests confirmed that all MVP requirements were fully satisfied. All ten system test scenarios and nine acceptance criteria resulted in a passing or accepted status, indicating that the BCanteen system meets its defined functional and quality requirements.